

將主畫面小工具新增至 Flutter 應用程式

來源：<https://codelabs.developers.google.com/flutter-home-screen-widgets?hl=zh-tw>

步驟數：7

在本程式碼研究室中，您將為 iOS 或 Android Flutter 應用程式建立主畫面小工具。我們會從基本的 Flutter 新聞應用程式開始說明。然後，您將使用原生架構為小工具本身建立 UI。最後，您將學到如何分享資源，以及建立小工具與主應用程式之間的通訊。

目錄

1. [1. 簡介](#)
2. [2. 設定開發環境](#)
3. [3. 新增基本主畫面小工具](#)
4. [4. 將 Flutter 應用程式中的資料傳送至主畫面小工具](#)
5. [5. 在 iOS 主畫面小工具中使用 Flutter 應用程式自訂字型](#)
6. [6. 將 Flutter 小工具算繪為圖片](#)
7. [7. 後續步驟](#)

步驟 1 · 約 5 分鐘

1. 簡介

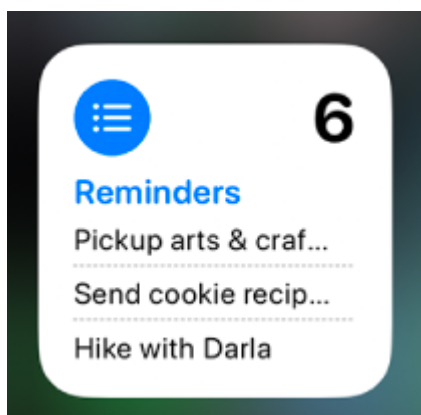
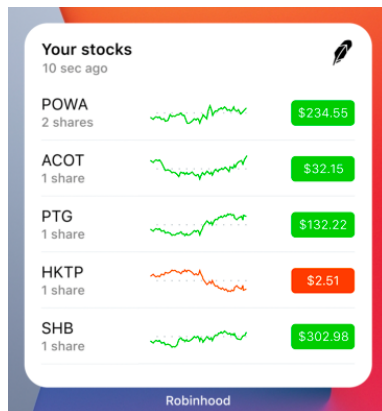
什麼是小工具？

對 Flutter 開發人員來說，*widget* 的常見定義是指使用 Flutter 架構建立的 UI 元件。在本程式碼研究室中，小工具是指應用程式的迷你版，可不必開啟應用程式就能查看應用程式資訊。在 Android 裝置上，小工具會顯示在主畫面上。在 iOS 裝置上，小工具可以新增至主畫面、螢幕鎖定畫面或「今天」檢視畫面。



小工具的複雜程度上限為何？

大多數主畫面小工具都很簡單，可能包含基本文字、簡單的圖像，或 Android 上的基本控制項。Android 和 iOS 都會限制可使用的 UI 元件和功能。



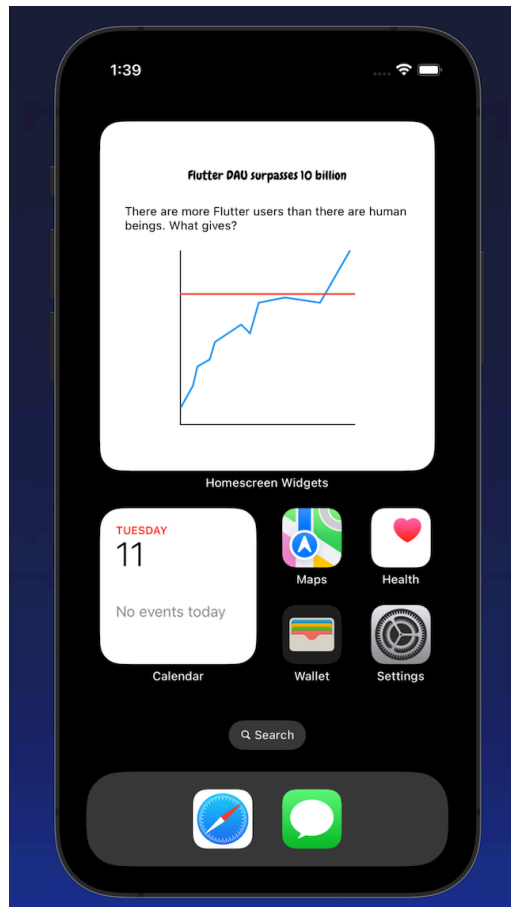
建立小工具的 UI

由於這些 UI 限制，您無法使用 Flutter 架構直接繪製主畫面小工具的 UI。您可以將使用 Jetpack Compose 或 SwiftUI 等平台框架建立的小工具新增至 Flutter 應用程式。本程式碼研究室將討論在應用程式和小工具之間共用資源的範例，避免重新編寫複雜的 UI。

建構項目

在本程式碼研究室中，您將使用 `home_widget` 套件，為簡單的 Flutter 應用程式在 Android 和 iOS 上建構主畫面小工具，讓使用者可以閱讀文章。小工具會：

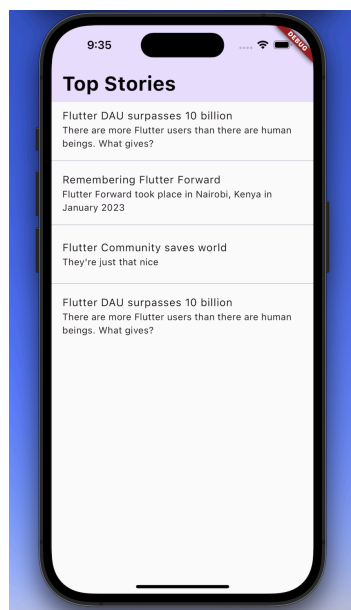
- 顯示 Flutter 應用程式的資料。
- 使用從 Flutter 應用程式共用的字型資產顯示文字。
- 顯示已算繪的 Flutter 小工具圖片。

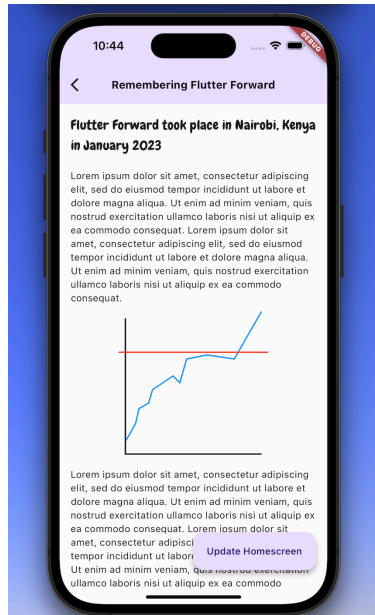


這個 Flutter 應用程式包含兩個畫面 (或路徑)：

- 第一個畫面會顯示新聞文章清單，內含廣告標題和說明。
- 第二個畫面顯示完整文章，以及使用 `CustomPaint` 建立的圖表。

。





學習目標

- 如何在 iOS 和 Android 裝置上建立主畫面小工具。
- 如何使用 `home_widget` 套件，在主畫面小工具和 Flutter 應用程式之間共用資料。
- 如何減少需要重新編寫的程式碼量。
- 如何從 Flutter 應用程式更新主畫面小工具。

注意：本程式碼研究室著重於建立主畫面小工具。並簡要說明 Flutter 應用程式程式碼。如要進一步瞭解如何開始使用 Flutter，請參閱「[您的第一個 Flutter 應用程式](#)」程式碼研究室。

步驟 2 · 約 5 分鐘

2. 設定開發環境

您需要 [Flutter SDK](#) 和 [IDE](#)。您可以使用偏好的 IDE 處理 Flutter。這可能是安裝 [Dart Code](#) 和 [Flutter 擴充功能](#) 的 Visual Studio Code，或是安裝 [Flutter](#) 和 [Dart 外掛程式](#) 的 Android Studio 或 IntelliJ。

如何建立 iOS 主畫面小工具：

- 您可以在實體 iOS 裝置或 iOS 模擬器上執行本程式碼研究室。
- 您必須使用 Xcode IDE 設定 macOS 系統。這會安裝建構 iOS 版應用程式所需的編譯器。

如何建立 Android 主畫面小工具：

- 您可以在實體 Android 裝置或 Android 模擬器上執行本程式碼研究室。
- 您必須使用 Android Studio 設定開發系統。這會安裝建構 Android 版應用程式所需的編譯器。

取得範例程式碼

從 GitHub 下載專案的初始版本

在指令列中，將 GitHub 存放區複製到 flutter-codelabs 目錄：

```
$ git clone https://github.com/flutter/codelabs.git flutter-codelabs
```

複製存放區後，您可以在 flutter-codelabs/homescreen_codelab 目錄中找到本程式碼研究室的程式碼。這個目錄包含程式碼研究室每個步驟的完整專案程式碼。

開啟範例應用程式

在想用的 IDE 中開啟 `flutter-codelabs/homescreen_codelab/step_03` 目錄。

安裝套件

所有必要套件都已新增至專案的 pubspec.yaml 檔案。如要擷取專案依附元件，請執行下列指令：

```
$ flutter pub get
```

3. 新增基本主畫面小工具

首先，請使用原生平台工具新增主畫面小工具。

注意：本程式碼研究室的 iOS 和 Android 部分彼此獨立，也與 Dart 程式碼無關。如果只對 iOS 感興趣，可以略過 Android 版本，反之亦然。

建立基本的 iOS 主畫面小工具

在 Flutter iOS 應用程式中新增應用程式擴充功能，與在 SwiftUI 或 UIKit 應用程式中新增應用程式擴充功能類似：

1. 在 Flutter 專案目錄的終端機視窗中執行 `open ios/Runner.xcworkspace`。或者，在 VSCode 中對「ios」資料夾按一下滑鼠右鍵，然後選取「Open in Xcode」。系統會隨即在 Flutter 專案中開啟預設的 Xcode 工作區。
2. 從選單中選取「File」→「New」→「Target」。這會在專案中新增目標。
3. 畫面上會顯示範本清單。選取「小工具擴充功能」。
4. 在「Product Name」（產品名稱）方塊中輸入這個小工具的名稱「NewsWidgets」。取消勾選「Include Live Activity」（包含即時活動）和「Include Configuration Intent」（包含設定意圖）核取方塊。

檢查程式碼範例

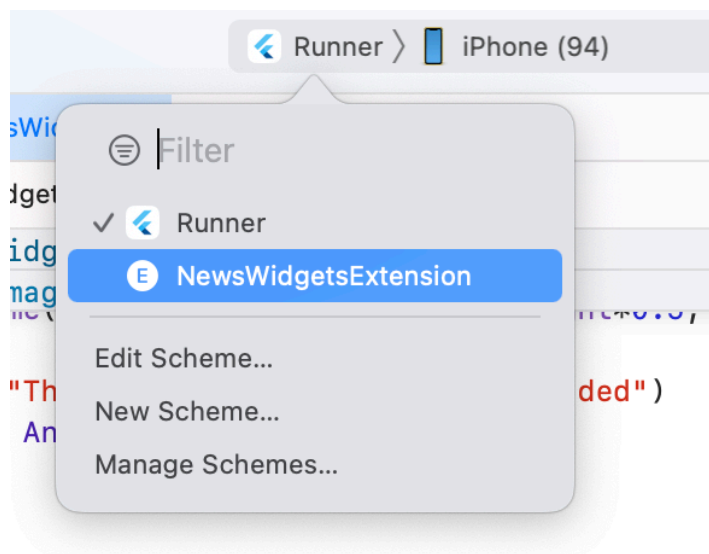
新增目標時，Xcode 會根據所選範本產生程式碼範例。如要進一步瞭解產生的程式碼和 WidgetKit，請參閱 [Apple 的應用程式擴充功能說明文件](#)。

偵錯及測試範例小工具

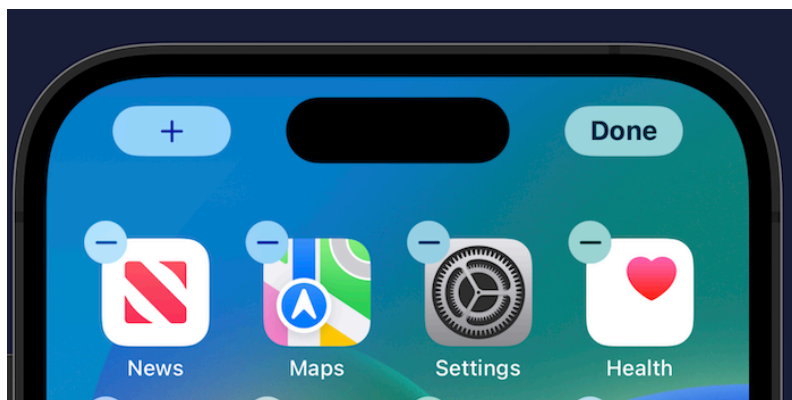
1. 首先，請更新 Flutter 應用程式的設定。在 Flutter 應用程式中新增套件，並打算從 Xcode 執行專案中的目標時，您必須執行這項操作。如要更新應用程式的設定，請在 Flutter 應用程式目錄中執行下列指令：

```
$ flutter build ios --config-only
```

2. 按一下「Runner」，開啟目標清單。選取剛建立的小工具目標 NewsWidgets，然後按一下「Run」。變更 iOS 小工具程式碼時，請從 Xcode 執行小工具目標。



3. 模擬器或裝置螢幕應顯示基本主畫面小工具。如果沒看到，可以新增至畫面。按住主畫面，然後按一下左上角的「+」。



4. 搜尋應用程式名稱。在本程式碼研究室中，請搜尋「Homescreen Widgets」

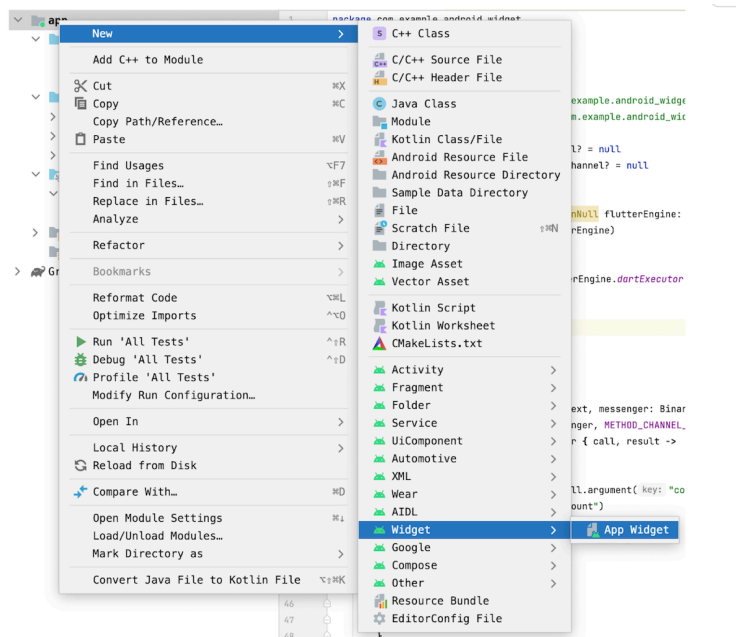


5. 新增主畫面小工具後，應該會顯示簡單的文字，指出時間。

注意：[Xcode 14.1 版本資訊](#)提到偵錯小工具時發生錯誤。您可以在 [StackOverflow](#) 找到實用的偵錯提示。

建立基本 Android 小工具

1. 如要在 Android 中新增主畫面小工具，請在 Android Studio 中開啟專案的建構檔案。這個檔案位於 `android/build.gradle`。或者，在 VSCode 中按一下滑鼠右鍵，選取「android」資料夾，然後選取「Open in Android Studio」。
2. 專案建構完成後，請在左上角找到應用程式目錄。將新的主畫面小工具新增至這個目錄。在目錄上按一下滑鼠右鍵，然後依序選取「New」->「Widget」->「App Widget」。



3. Android Studio 會顯示新表單。新增主畫面小工具的基本資訊，包括類別名稱、位置、大小和原文語言

請為這個程式碼研究室設定下列值：

- 將「類別名稱」方塊改為 `NewsWidget`
- 將「最小寬度 (儲存格)」下拉式選單設為 3
- 將「最低高度 (儲存格)」下拉式選單設為 3

注意：高度和小工具屬性會決定小工具在主畫面佔用的空間。Android 會以儲存格表示這些項目。一個儲存格等於應用程式圖示可佔用的空間。之後可以變更這個值。

檢查程式碼範例

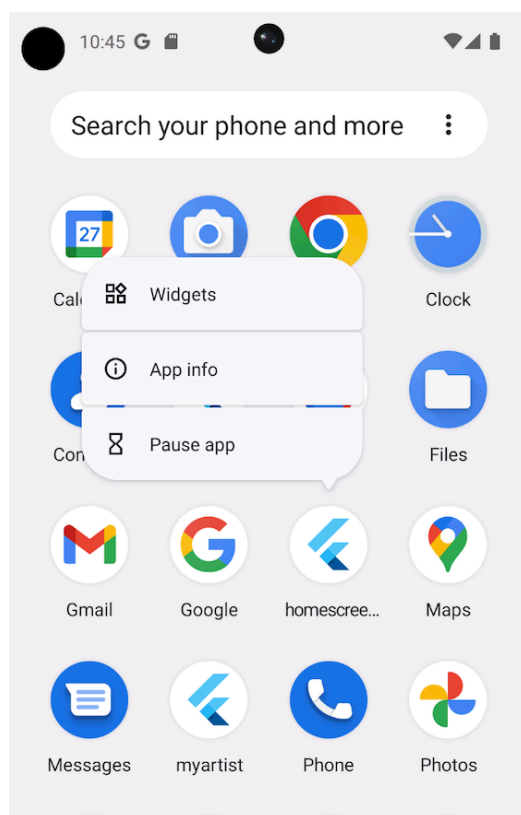
提交表單後，Android Studio 會建立及更新多個檔案。下表列出與本程式碼研究室相關的變更

動作	目標檔案	變更
更新	<code>AndroidManifest.xml</code>	新增註冊 <code>NewsWidget</code> 的接收器。
建立	<code>res/layout/news_widget.xml</code>	定義主畫面小工具 UI。
建立	<code>res/xml/news_widget_info.xml</code>	定義主畫面小工具設定。您可以在這個檔案中調整小工具的尺寸或名稱。
建立	<code>java/com/example/homescreen_widgets/NewsWidget.kt</code>	包含 Kotlin 程式碼，可為主畫面小工具新增功能。

在本程式碼研究室中，您會進一步瞭解這些檔案。

偵錯及測試範例小工具

現在執行應用程式，即可看到主畫面小工具。建構應用程式後，請前往 Android 裝置的應用程式選取畫面，然後長按這個 Flutter 專案的圖示。在彈出式選單中選取「小工具」。



Android 裝置或模擬器會顯示 Android 的預設主畫面小工具。

步驟 4 · 約 15 分鐘

4. 將 Flutter 應用程式中的資料傳送至主畫面小工具

你可以自訂建立的基本主畫面小工具。更新主畫面小工具，顯示新聞報導的標題和摘要。以下螢幕截圖顯示主畫面小工具的範例，當中會顯示新聞標題和摘要。



如要在應用程式和主畫面小工具之間傳遞資料，您需要編寫 Dart 和原生程式碼。本節會將這項程序分為三部分：

1. 在 Flutter 應用程式中編寫 Android 和 iOS 皆可使用的 Dart 程式碼
2. 新增原生 iOS 功能
3. 新增原生 Android 功能

使用 iOS 應用程式群組

如要在 iOS 父項應用程式和小工具擴充功能之間共用資料，這兩個目標必須屬於同一個應用程式群組。如要進一步瞭解應用程式群組，請參閱 [Apple 的應用程式群組說明文件](#)。

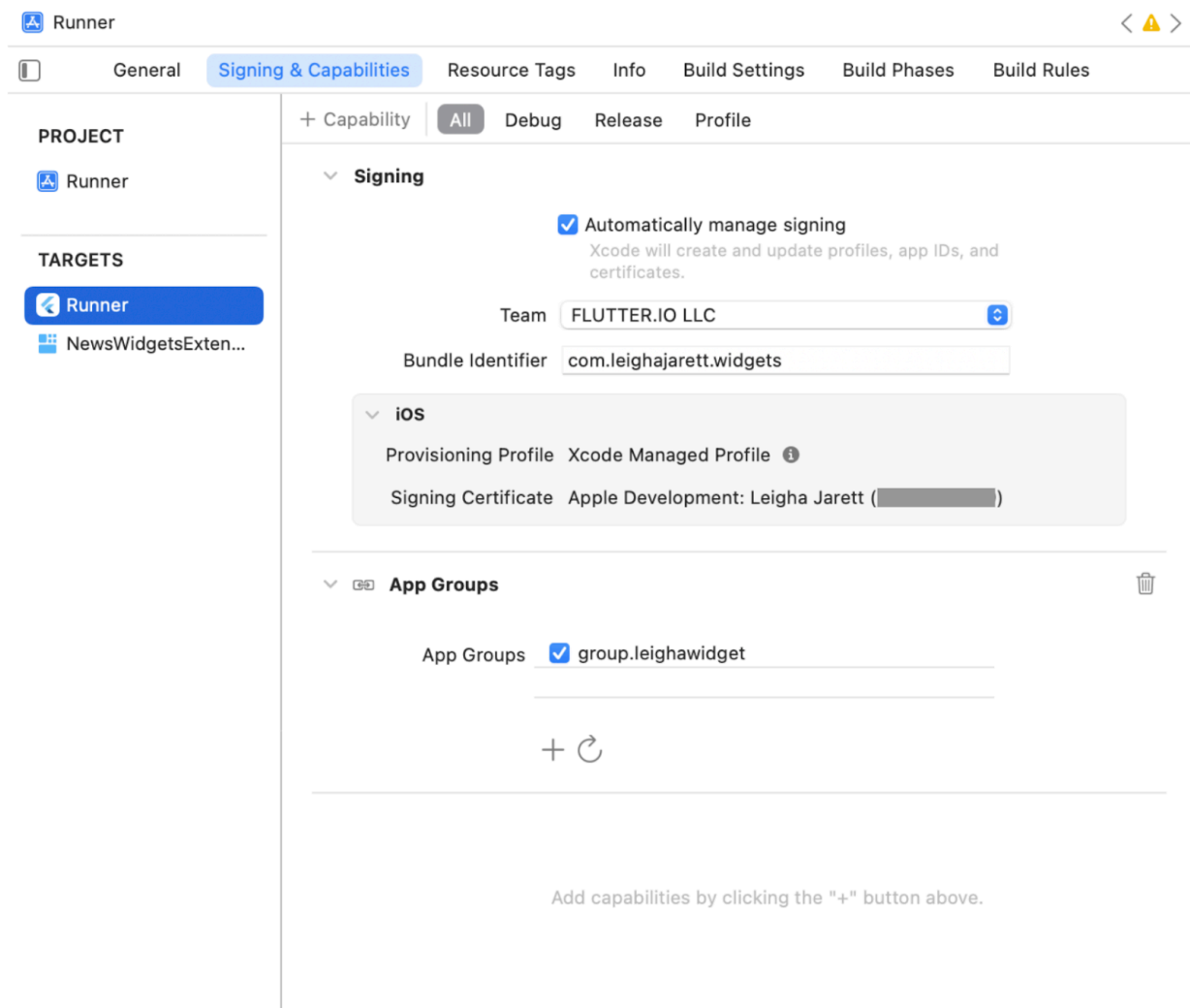
注意：您必須登入 Apple 開發人員帳戶，才能新增應用程式群組。

更新軟體包 ID：

在 Xcode 中，前往目標的設定。在「Signing & Capabilities」分頁中，確認團隊和軟體包 ID 已設定。

在 Xcode 中，將 App Group 同時新增至 Runner 目標和 NewsWidgetExtension 目標：

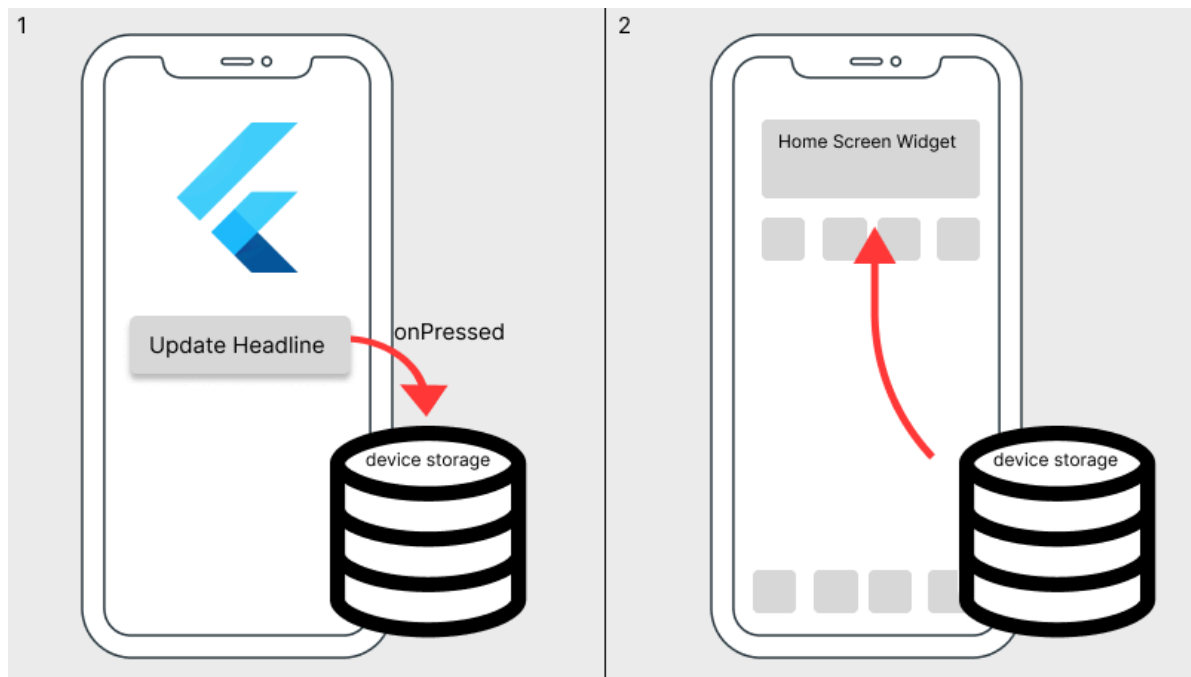
選取「+ Capability」->「App Groups」，然後新增應用程式群組。針對 Runner (父項應用程式) 目標和小工具目標，重複上述步驟。



注意：小工具的軟體包 ID 必須使用父項應用程式的軟體包 ID 做為前置字元。

新增 Dart 程式碼

iOS 和 Android 應用程式可透過幾種不同方式與 Flutter 應用程式共用資料。如要與這些應用程式通訊，請善用裝置的本機 key/value 儲存庫。iOS 將這個儲存庫稱為 `UserDefaults`，Android 則稱為 `SharedPreferences`。 [home_widget 套件](#) 會包裝這些 API，簡化將資料儲存至任一平台的方式，並讓主畫面小工具提取更新的資料。



廣告標題和說明資料來自 `news_data.dart` 檔案。這個檔案包含模擬資料和 `NewsArticle` 資料類別。

[lib/news_data.dart](#)

```
class NewsArticle {  
  final String title;  
  final String description;  
  final String? articleText;  
  
  NewsArticle({  
    required this.title,  
    required this.description,  
    this.articleText = loremIpsum,  
  });  
}
```

更新廣告標題和說明值

如要新增從 Flutter 應用程式更新主畫面小工具的功能，請前往 `lib/home_screen.dart` 檔案。將檔案內容替換成下列程式碼。然後將 `<YOUR_APP_GROUP>` 替換為應用程式群組的 ID。

[lib/home_screen.dart](#)

```

import 'package:flutter/material.dart';
import 'package:home_widget/home_widget.dart';           // Add this import

import 'article_screen.dart';
import 'news_data.dart';

// TODO: Replace with your App Group ID
const String appId = '<YOUR APP GROUP>';                 // Add from here
const String iosWidgetName = 'NewsWidgets';
const String androidWidgetName = 'NewsWidget';          // To here.

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key});
  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

void updateHeadline(NewsArticle newHeadline) {           // Add from here
  // Save the headline data to the widget
  HomeWidget.saveWidgetData<String>('headline_title', newHeadline.title);
  HomeWidget.saveWidgetData<String>('headline_description', newHeadline.description);
  HomeWidget.updateWidget(
    iosName: iosWidgetName,
    androidName: androidWidgetName,
  );
}                                                         // To here.

class _MyHomePageState extends State<MyHomePage> {

  @override                                               // Add from here
  void initState() {
    super.initState();

    HomeWidget.setAppGroupId(appId);

    // Mock read in some data and update the headline
    final newHeadline = getNewsStories()[0];
    updateHeadline(newHeadline);
  }                                                         // To here.

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Top Stories'),
        centerTitle: false,
        titleTextStyle: const TextStyle(
          fontSize: 30,
          fontWeight: FontWeight.bold,
          color: Colors.black)),
      body: ListView.separated(

```

```

        separatorBuilder: (context, idx) {
          return const Divider();
        },
        itemCount: getNewsStories().length,
        itemBuilder: (context, idx) {
          final article = getNewsStories()[idx];
          return ListTile(
            key: Key('$idx ${article.hashCode}'),
            title: Text(article.title!),
            subtitle: Text(article.description!),
            onTap: () {
              Navigator.of(context).push(
                MaterialPageRoute(
                  builder: (context) {
                    return ArticleScreen(article: article);
                  },
                ),
              );
            },
          );
        },
      );
    },
  ));
}
}

```

`updateHeadline` 函式會將鍵/值配對儲存至裝置的本機儲存空間。`headline_title` 鍵會保留 `newHeadline.title` 的值。`headline_description` 鍵會保留 `newHeadline.description` 的值。這個函式也會通知原生平台，可以擷取並算繪主畫面小工具的新資料。

修改 `floatingActionButton`

按下 `floatingActionButton` 時呼叫 `updateHeadline` 函式，如下所示：

[lib/article_screen.dart](#)

```

// New: import the updateHeadline function
import 'home_screen.dart';

...

floatingActionButton: FloatingActionButton.extended(
  onPressed: () {
    ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
      content: Text('Updating home screen widget...'),
    ));
    // New: call updateHeadline
    updateHeadline(widget.article);
  },
  label: const Text('Update Homescreen'),
),
...

```

這項異動生效後，使用者在文章頁面按下「更新廣告標題」按鈕時，主畫面小工具詳細資料就會更新。

注意：如果裝置的本機儲存空間中已有這些鍵，這個小工具會以新值覆寫目前的值。

更新 iOS 程式碼，顯示文章資料

如要更新 iOS 的主畫面小工具，請使用 Xcode。

在 Xcode 中開啟 `NewsWidgets.swift` 檔案：

設定 `TimelineEntry`。

使用下列程式碼取代 `SimpleEntry` 結構體：

ios/NewsWidgets/NewsWidgets.swift

```
// The date and any data you want to pass into your app must conform to TimelineEntry
struct NewsArticleEntry: TimelineEntry {
    let date: Date
    let title: String
    let description:String
}
```

這個 `NewsArticleEntry` 結構體會定義要傳遞至主畫面小工具的傳入資料 (更新時)。`TimelineEntry` 類型需要日期參數。如要進一步瞭解 `TimelineEntry` 通訊協定，請參閱 [Apple 的 TimelineEntry 說明文件](#)。

編輯 view 顯示內容

修改主畫面小工具，改為顯示新聞報導的標題和說明，而非日期。如要在 SwiftUI 中顯示文字，請使用 `Text` 檢視區塊。如要在 SwiftUI 中將檢視區塊彼此堆疊，請使用 `VStack` 檢視區塊。

使用下列程式碼取代產生的 `NewsWidgetEntryView` 檢視區塊：

ios/NewsWidgets/NewsWidgets.swift

```
//View that holds the contents of the widget
struct NewsWidgetsEntryView : View {
    var entry: Provider.Entry

    var body: some View {
        VStack {
            Text(entry.title)
            Text(entry.description)
        }
    }
}
```

編輯供應器，告知主畫面小工具更新時間和方式

將現有的 `Provider` 替換成下列程式碼。然後，將 `<YOUR APP GROUP>` 替換為應用程式群組 ID：

ios/NewsWidgets/NewsWidgets.swift

```
struct Provider: TimelineProvider {

    // Placeholder is used as a placeholder when the widget is first displayed
    func placeholder(in context: Context) -> NewsArticleEntry {
        //      Add some placeholder title and description, and get the current date
        NewsArticleEntry(date: Date(), title: "Placeholder Title", description: "Placeholder
description")
    }

    // Snapshot entry represents the current time and state
    func getSnapshot(in context: Context, completion: @escaping (NewsArticleEntry) -> ()) {
        let entry: NewsArticleEntry
        if context.isPreview{
            entry = placeholder(in: context)
        }
        else{
            //      Get the data from the user defaults to display
            let userDefaults = UserDefaults(suiteName: <YOUR APP GROUP>)
            let title = userDefaults?.string(forKey: "headline_title") ?? "No Title Set"
            let description = userDefaults?.string(forKey: "headline_description") ?? "No
Description Set"
            entry = NewsArticleEntry(date: Date(), title: title, description: description)
        }
        completion(entry)
    }

    //      getTimeline is called for the current and optionally future times to update the widget
    func getTimeline(in context: Context, completion: @escaping (Timeline<Entry>) -> ()) {
        //      This just uses the snapshot function you defined earlier
        getSnapshot(in: context) { (entry) in
            // atEnd policy tells widgetkit to request a new entry after the date has passed
            let timeline = Timeline(entries: [entry], policy: .atEnd)
            completion(timeline)
        }
    }
}
```

先前程式碼中的 `Provider` 符合 `TimelineProvider` 。 `Provider` 有三種不同的方法：

1. 使用者首次預覽主畫面小工具時， `placeholder` 方法會產生預留位置項目。



2. `getSnapshot` 方法會從使用者預設值讀取資料，並產生目前時間的項目。
3. `getTimeline` 方法會傳回時間軸項目。如果內容更新時間可預測，這項功能就很有幫助。本程式碼研究室會使用 `getSnapshot` 函式取得目前狀態。`.atEnd` 方法會告知主畫面小工具，目前時間過後要重新整理資料。

註解掉 `NewsWidgets_Previews`

本程式碼研究室不會說明如何使用預覽畫面。如要進一步瞭解如何預覽 SwiftUI 主畫面小工具，請參閱 [Apple 的「偵錯小工具」說明文件](#)。

儲存所有檔案，然後重新執行應用程式和小工具目標。

再次執行目標，驗證應用程式和主畫面小工具是否正常運作。

1. 在 Xcode 中選取應用程式結構定義，執行應用程式目標。
2. 在 Xcode 中選取擴充功能結構定義，執行擴充功能目標。
3. 在應用程式中前往文章頁面。
4. 按一下按鈕即可更新標題。主畫面小工具的標題也應會更新。

更新 Android 程式碼

新增主畫面小工具 XML。

在 Android Studio 中，更新上一個步驟產生的檔案。開啟 `res/layout/news_widget.xml` 檔案。定義主畫面小工具的結構和版面配置。選取右上角的「程式碼」，然後將該檔案的內容替換成下列程式碼：

[android/app/res/layout/news_widget.xml](#)

```

<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/widget_container"
    style="@style/Widget.Android.AppWidget.Container"
    android:layout_width="wrap_content"
    android:layout_height="match_parent"
    android:background="@android:color/white"
    android:theme="@style/Theme.Android.AppWidgetContainer">

    <TextView
        android:id="@+id/headline_title"
        style="@style/Widget.Android.AppWidget.InnerView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentStart="true"
        android:layout_alignParentLeft="true"
        android:layout_marginStart="8dp"
        android:layout_marginLeft="8dp"
        android:background="@android:color/white"
        android:text="Title"
        android:textSize="20sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/headline_description"
        style="@style/Widget.Android.AppWidget.InnerView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@+id/headline_title"
        android:layout_alignParentStart="true"
        android:layout_alignParentLeft="true"
        android:layout_marginStart="8dp"
        android:layout_marginLeft="8dp"
        android:layout_marginTop="4dp"
        android:background="@android:color/white"
        android:text="Title"
        android:textSize="16sp" />

</RelativeLayout>

```

這個 XML 定義了兩個文字檢視區塊，一個用於文章標題，另一個用於文章說明。這些文字檢視區塊也會定義樣式。在本程式碼研究室中，您會多次返回這個檔案。

注意：這個 XML 檔案屬於標準 Android 程式碼集。這個函式會定義主畫面小工具的 UI，但詳細說明其功能超出本程式碼研究室的範圍。如要進一步瞭解 Android 開發，請參閱 [Android 開發人員指南說明文件](#)。

更新 NewsWidget 功能

開啟 `NewsWidget.kt` Kotlin 原始碼檔案。這個檔案包含名為 `NewsWidget` 的產生類別，可擴充 `AppWidgetProvider` 類別。

`NewsWidget` 類別包含來自父類別的三個方法。您將修改 `onUpdate` 方法。Android 會以固定間隔呼叫這個方法，更新小工具。

注意：Android 會在固定時間間隔呼叫 `onUpdate`，要求小工具「自行更新」。預設間隔為 86,400,000 毫秒，或 24 小時。您也可以透過程式執行這項操作。當您在 Dart 程式碼中呼叫 `updateWidget` 時，`home_widget` 套件會告知 Android 呼叫 `onUpdate`。

將 `NewsWidget.kt` 檔案的內容替換成以下程式碼：

android/app/java/com.mydomain.homescreen_widgets/NewsWidget.kt

```

// Import will depend on App ID.
package com.mydomain.homescreen_widgets

import android.appwidget.AppWidgetManager
import android.appwidget.AppWidgetProvider
import android.content.Context
import android.widget.RemoteViews

// New import.
import es.antonborri.home_widget.HomeWidgetPlugin

/**
 * Implementation of App Widget functionality.
 */
class NewsWidget : AppWidgetProvider() {
    override fun onUpdate(
        context: Context,
        appWidgetManager: AppWidgetManager,
        appWidgetIds: IntArray,
    ) {
        for (appWidgetId in appWidgetIds) {
            // Get reference to SharedPreferences
            val widgetData = HomeWidgetPlugin.getData(context)
            val views = RemoteViews(context.packageName, R.layout.news_widget).apply {

                val title = widgetData.getString("headline_title", null)
                setTextViewText(R.id.headline_title, title ?: "No title set")

                val description = widgetData.getString("headline_description", null)
                setTextViewText(R.id.headline_description, description ?: "No description
set")
            }

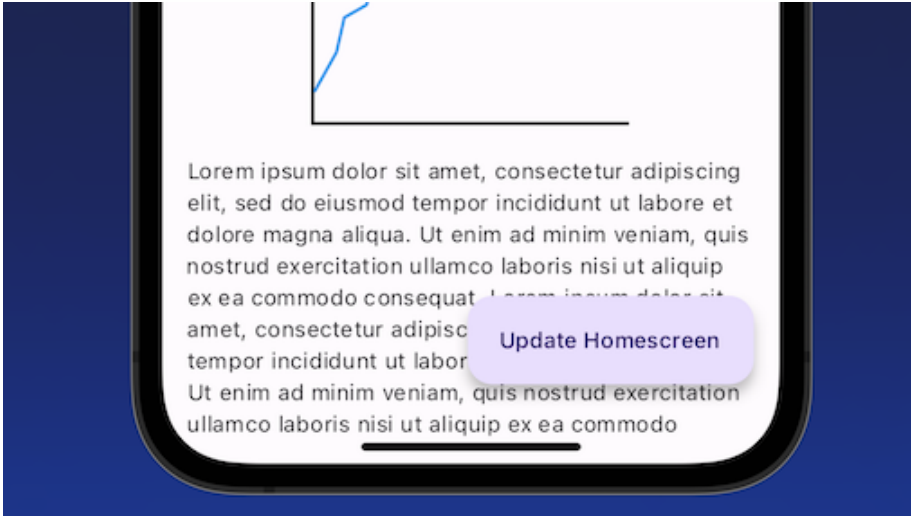
            appWidgetManager.updateAppWidget(appWidgetId, views)
        }
    }
}

```

現在呼叫 `onUpdate` 時，Android 會使用 `the widgetData.getString()` 方法從本機儲存空間取得最新值，然後呼叫 `setTextViewText` 變更主畫面小工具上顯示的文字。

測試更新

測試應用程式，確保主畫面小工具會更新新資料。如要更新資料，請使用文章頁面上的「更新主畫面」`FloatingActionButton`。主畫面小工具應會更新文章標題。



5. 在 iOS 主畫面小工具中使用 Flutter 應用程式自訂字型

到目前為止，您已設定主畫面小工具，讀取 Flutter 應用程式提供的資料。Flutter 應用程式包含您可能想在主畫面小工具中使用的自訂字型。您可以在 iOS 主畫面小工具中使用自訂字型。Android 不支援在主畫面小工具中使用自訂字型。

更新 iOS 程式碼

Flutter 會將資產儲存在 iOS 應用程式的 mainBundle 中。您可以從主畫面小工具程式碼存取這個套件中的資產。

在 NewsWidgets.swift 檔案的 NewsWidgetsEntryView 結構體中，進行下列變更：

建立輔助函式，取得 Flutter 資產目錄的路徑：

ios/NewsWidgets/NewsWidgets.swift

```
struct NewsWidgetsEntryView : View {  
    ...  
  
    // New: Add the helper function.  
    var bundle: URL {  
        let bundle = Bundle.main  
        if bundle.bundleURL.pathExtension == "appex" {  
            // Peel off two directory levels - MY_APP.app/PlugIns/MY_APP_EXTENSION.appex  
            var url =  
bundle.bundleURL.deletingLastPathComponent().deletingLastPathComponent()  
            url.append(component: "Frameworks/App.framework/flutter_assets")  
            return url  
        }  
        return bundle.bundleURL  
    }  
    ...  
}
```

使用自訂字型檔案的網址註冊字型。

ios/NewsWidgets/NewsWidgets.swift

```
struct NewsWidgetsEntryView : View {  
    ...  
  
    // New: Register the font.  
    init(entry: Provider.Entry){  
        self.entry = entry  
        CTFontManagerRegisterFontsForURL(bundle.appending(path: "/fonts/Chewy-Regular.ttf") as  
CFURL, CTFontManagerScope.process, nil)  
    }  
    ...  
}
```

更新標題的文字檢視區塊，使用自訂字型。

[ios/NewsWidgets/NewsWidgets.swift](#)

```
struct NewsWidgetsEntryView : View {  
    ...  
  
    var body: some View {  
        VStack {  
            // Update the following line.  
            Text(entry.title).font(Font.custom("Chewy", size: 13))  
            Text(entry.description)  
        }  
    }  
    ...  
}
```

執行主畫面小工具時，系統會使用自訂字型顯示標題，如下圖所示：



6. 將 Flutter 小工具算繪為圖片

在本節中，您要將 Flutter 應用程式中的圖表顯示為主畫面小工具。

這個小工具比主畫面上的文字更具挑戰性。與其嘗試使用原生 UI 元件重新建立 Flutter 圖表，不如將圖表顯示為圖片，這樣簡單多了。

編寫主畫面小工具的程式碼，將 Flutter 圖表算繪為 PNG 檔案。主畫面小工具可以顯示該圖片。

注意：如要進一步瞭解 SwiftUI Views 如何對應至 Flutter 小工具，請參閱「[以 SwiftUI 開發人員的身分學習 Flutter](#)」。如要進一步瞭解 Android 框架如何對應至 Flutter 小工具，請參閱「[以 Android 開發人員身分學習 Flutter](#)」。

編寫 Dart 程式碼

在 Dart 端，從 `home_widget` 套件新增 `renderFlutterWidget` 方法。這個方法會採用小工具、檔案名稱和金鑰。並傳回 Flutter 小工具的圖片，儲存至共用容器。在程式碼中提供圖片名稱，並確保主畫面小工具可以存取容器。`key` 會將完整檔案路徑儲存為裝置本機儲存空間中的字串。這樣一來，即使 Dart 程式碼中的名稱有所變更，主畫面小工具也能找到檔案。

在本程式碼研究室中，`lib/article_screen.dart` 檔案中的 `LineChart` 類別代表圖表。建構方法會傳回 `CustomPainter`，將這個圖表繪製到畫面上。

如要實作這項功能，請開啟 `lib/article_screen.dart` 檔案。匯入 `home_widget` 套件。接著，將 `_ArticleScreenState` 類別中的程式碼替換為下列程式碼：

[lib/article_screen.dart](#)

```

import 'package:flutter/material.dart';
// New: import the home_widget package.
import 'package:home_widget/home_widget.dart';

import 'home_screen.dart';
import 'news_data.dart';

...

class _ArticleScreenState extends State<ArticleScreen> {
  // New: add this GlobalKey
  final _globalKey = GlobalKey();
  String? imagePath;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.article.title!),
      ),
      // New: add this FloatingActionButton
      floatingActionButton: FloatingActionButton.extended(
        onPressed: () async {
          if (_globalKey.currentContext != null) {
            var path = await HomeWidget.renderFlutterWidget(
              const LineChart(),
              fileName: 'screenshot',
              key: 'filename',
              logicalSize: _globalKey.currentContext!.size,
              pixelRatio:
                MediaQuery.of(_globalKey.currentContext!).devicePixelRatio,
            );
            setState(() {
              imagePath = path as String?;
            });
          }
          updateHeadline(widget.article);
        },
        label: const Text('Update Homescreen'),
      ),
      body: ListView(
        padding: const EdgeInsets.all(16.0),
        children: [
          Text(
            widget.article.description!,
            style: Theme.of(context).textTheme.titleMedium,
          ),
          const SizedBox(height: 20.0),
          Text(widget.article.articleText!),
          const SizedBox(height: 20.0),
          Center(
            // New: Add this key

```

```

        key: _globalKey,
        child: const LineChart(),
      ),
      const SizedBox(height: 20.0),
      Text(widget.article.articleText!),
    ],
  ),
);
}
}

```

這個範例會對 `_ArticleScreenState` 類別進行三項變更。

建立 GlobalKey

`GlobalKey` 會取得特定小工具的情境資訊，這是取得該小工具大小的必要條件。

lib/article_screen.dart

```

class _ArticleScreenState extends State<ArticleScreen> {
  // New: add this GlobalKey
  final _globalKey = GlobalKey();
  ...
}

```

新增 imagePath

`imagePath` 屬性會儲存 Flutter 小工具的圖片位置。

lib/article_screen.dart

```

class _ArticleScreenState extends State<ArticleScreen> {
  ...
  // New: add this imagePath
  String? imagePath;
  ...
}

```

將鍵新增至要算繪的小工具

`_globalKey` 包含要算繪成圖片的 Flutter 小工具。在本例中，Flutter 小工具是包含 `LineChart` 的 `Center`。

lib/article_screen.dart

```
class _ArticleScreenState extends State<ArticleScreen> {
  ...
  Center(
    // New: Add this key
    key: _globalKey,
    child: const LineChart(),
  ),
  ...
}
```

4. 將小工具儲存為圖片

使用者點選 `floatingActionButton` 時，系統會呼叫 `renderFlutterWidget` 方法。這個方法會將產生的 PNG 檔案儲存至共用容器目錄，並命名為「screenshot」。這個方法也會將圖片的完整路徑儲存為裝置儲存空間中的檔案名稱鍵。

lib/article_screen.dart

```
class _ArticleScreenState extends State<ArticleScreen> {
  ...
  floatingActionButton: FloatingActionButton.extended(
    onPressed: () async {
      if (_globalKey.currentContext != null) {
        var path = await HomeWidget.renderFlutterWidget(
          LineChart(),
          fileName: 'screenshot',
          key: 'filename',
          logicalSize: _globalKey.currentContext!.size,
          pixelRatio:
            MediaQuery.of(_globalKey.currentContext!).devicePixelRatio,
        );
        setState(() {
          imagePath = path as String?;
        });
      }
      updateHeadline(widget.article);
    },
    ...
  )
}
```

更新 iOS 程式碼

如果是 iOS，請更新程式碼，從儲存空間取得檔案路徑，並使用 SwiftUI 將檔案顯示為圖片。

開啟 `NewsWidgets.swift` 檔案，然後進行下列變更：

將 `filename` 和 `displaySize` 新增至 `NewsArticleEntry` struct

`filename` 屬性會保留代表圖片檔案路徑的字串。`displaySize` 屬性會保留使用者裝置上主畫面小工具的大小。主畫面小工具的大小來自 `context`。

ios/NewsWidgets/NewsWidgets.swift

```
struct NewsArticleEntry: TimelineEntry {
    ...

    // New: add the filename and displaySize.
    let filename: String
    let displaySize: CGSize
}
```

更新 `placeholder` 函式

加入預留位置 `filename` 和 `displaySize` 。

[ios/NewsWidgets/NewsWidgets.swift](#)

```
func placeholder(in context: Context) -> NewsArticleEntry {
    NewsArticleEntry(date: Date(), title: "Placeholder Title", description: "Placeholder
description", filename: "No screenshot available", displaySize: context.displaySize)
}
```

從 `getSnapshot` 中的 `userDefaults` 取得檔案名稱

當主畫面小工具更新時，這會將 `filename` 變數設為 `userDefaults` 儲存空間中的 `filename` 值。

[ios/NewsWidgets/NewsWidgets.swift](#)

```
func getSnapshot(
    ...

    let title = userDefaults?.string(forKey: "headline_title") ?? "No Title Set"
    let description = userDefaults?.string(forKey: "headline_description") ?? "No Description
Set"
    // New: get fileName from key/value store
    let filename = userDefaults?.string(forKey: "filename") ?? "No screenshot available"
    ...
)
```

建立 `ChartImage`，顯示路徑中的圖片

`ChartImage` `View` 會根據 Dart 端產生的檔案內容建立圖片。在此，您將大小設為影格的 50%。

[ios/NewsWidgets/NewsWidgets.swift](#)

```

struct NewsWidgetsEntryView : View {
  ...

  // New: create the ChartImage view
  var ChartImage: some View {
    if let uiImage = UIImage(contentsOfFile: entry.filename) {
      let image = Image(uiImage: uiImage)
        .resizable()
        .frame(width: entry.displaySize.height*0.5, height:
entry.displaySize.height*0.5, alignment: .center)
      return AnyView(image)
    }
    print("The image file could not be loaded")
    return AnyView(EmptyView())
  }
  ...
}

```

在 NewsWidgetsEntryView 的主體中使用 ChartImage。

將 ChartImage 檢視區塊新增至 NewsWidgetsEntryView 的主體，在主畫面小工具中顯示 ChartImage。

ios/NewsWidgets/NewsWidgets.swift

```

VStack {
  Text(entry.title).font(Font.custom("Chewy", size: 13))
  Text(entry.description).font(.system(size: 12)).padding(10)
  // New: add the ChartImage to the NewsWidgetEntryView
  ChartImage
}

```

測試變更

如要測試變更，請從 Xcode 重新執行 Flutter 應用程式 (Runner) 目標和擴充功能目標。如要查看圖片，請在應用程式中前往其中一個文章頁面，然後按下按鈕更新主畫面小工具。



更新 Android 程式碼

Android 程式碼的功能與 iOS 程式碼相同。

1. 開啟 `android/app/res/layout/news_widget.xml` 檔案，其中包含主畫面小工具的 UI 元素。將其內容替換為下列程式碼：

[android/app/res/layout/news_widget.xml](#)

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/widget_container"
    style="@style/Widget.Android.AppWidget.Container"
    android:layout_width="wrap_content"
    android:layout_height="match_parent"
    android:background="@android:color/white"
    android:theme="@style/Theme.Android.AppWidgetContainer">
```

```
<TextView
    android:id="@+id/headline_title"
    style="@style/Widget.Android.AppWidget.InnerView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignParentStart="true"
    android:layout_alignParentLeft="true"
    android:layout_marginStart="8dp"
    android:layout_marginLeft="8dp"
    android:background="@android:color/white"
    android:text="Title"
    android:textSize="20sp"
    android:textStyle="bold" />
```

```
<TextView
    android:id="@+id/headline_description"
    style="@style/Widget.Android.AppWidget.InnerView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@+id/headline_title"
    android:layout_alignParentStart="true"
    android:layout_alignParentLeft="true"
    android:layout_marginStart="8dp"
    android:layout_marginLeft="8dp"
    android:layout_marginTop="4dp"
    android:background="@android:color/white"
    android:text="Title"
    android:textSize="16sp" />
```

```
<!--New: add this image view -->
```

```
<ImageView
    android:id="@+id/widget_image"
    android:layout_width="200dp"
    android:layout_height="200dp"
    android:layout_below="@+id/headline_description"
    android:layout_alignBottom="@+id/headline_title"
    android:layout_alignParentStart="true"
    android:layout_alignParentLeft="true"
    android:layout_marginStart="8dp"
    android:layout_marginLeft="8dp"
    android:layout_marginTop="6dp"
    android:layout_marginBottom="-134dp"
```

```
        android:layout_weight="1"
        android:adjustViewBounds="true"
        android:background="@android:color/white"
        android:scaleType="fitCenter"
        android:src="@android:drawable/star_big_on"
        android:visibility="visible"
        tools:visibility="visible" />
```

```
</RelativeLayout>
```

這段新程式碼會將圖片新增至主畫面小工具，目前會顯示一般星號圖示。將這個星號圖示換成您在 Dart 程式碼中儲存的圖片。

2. 開啟 `NewsWidget.kt` 檔案，將其內容替換為下列程式碼：

[android/app/java/com.mydomain.homescreen_widgets/NewsWidget.kt](#)

```

// Import will depend on App ID.
package com.mydomain.homescreen_widgets

import android.appwidget.AppWidgetManager
import android.appwidget.AppWidgetProvider
import android.content.Context
import android.graphics.Bitmap
import android.graphics.BitmapFactory
import android.widget.RemoteViews
import java.io.File
import es.antonborri.home_widget.HomeWidgetPlugin

/**
 * Implementation of App Widget functionality.
 */
class NewsWidget : AppWidgetProvider() {
    override fun onUpdate(
        context: Context,
        appWidgetManager: AppWidgetManager,
        appWidgetIds: IntArray,
    ) {
        for (appWidgetId in appWidgetIds) {
            val widgetData = HomeWidgetPlugin.getData(context)
            val views = RemoteViews(context.packageName, R.layout.news_widget).apply {

                val title = widgetData.getString("headline_title", null)
                setTextViewText(R.id.headline_title, title ?: "No title set")

                val description = widgetData.getString("headline_description", null)
                setTextViewText(R.id.headline_description, description ?: "No description
set")

                // New: Add the section below
                // Get chart image and put it in the widget, if it exists
                val imageName = widgetData.getString("filename", null)
                val imageFile = File(imageName)
                val imageExists = imageFile.exists()
                if (imageExists) {
                    val myBitmap: Bitmap = BitmapFactory.decodeFile(imageFile.absolutePath)
                    setImageViewBitmap(R.id.widget_image, myBitmap)
                } else {
                    println("image not found!, looked @: ${imageName}")
                }
                // End new code
            }

            appWidgetManager.updateAppWidget(appWidgetId, views)
        }
    }
}

```

這段 Dart 程式碼會使用 `filename` 鍵，將螢幕截圖儲存至本機儲存空間。此外，它也會取得圖片的完整路徑，並從中建立 `File` 物件。如果圖片存在，Dart 程式碼會以新圖片取代主畫面小工具中的圖片。

3. 重新載入應用程式，然後前往文章畫面。按一下「更新主畫面」。主畫面小工具會顯示圖表。
-

步驟 7 · 約 0 分鐘

7. 後續步驟

恭喜！

恭喜！您已成功為 Flutter iOS 和 Android 應用程式建立主畫面小工具！

連結至 Flutter 應用程式中的內容

您可能會想根據使用者點選的位置，將他們導向應用程式中的特定頁面。舉例來說，在本程式碼研究室的新聞應用程式中，您可能會希望使用者看到所顯示新聞標題的相關報導。

這項功能不在本程式碼研究室的討論範圍內。您可以查看[這個串流的使用範例 \(由 home_widget 套件提供\)](#)，瞭解如何從主畫面小工具啟動應用程式，以及透過網址從主畫面小工具傳送訊息。詳情請參閱 docs.flutter.dev 上的[深層連結說明文件](#)。

在背景更新小工具

在本程式碼研究室中，您使用按鈕觸發了主畫面小工具的更新。雖然這對測試來說很合理，但在正式版程式碼中，您可能會希望應用程式在背景更新主畫面小工具。您可以使用 workmanager 外掛程式建立背景工作，更新主畫面小工具所需的資源。如要瞭解詳情，請參閱 home_widget 套件中的「[背景更新](#)」一節。

如果是 iOS，你也可以讓主畫面小工具發出網路要求，更新其 UI。如要控管該要求的條件或頻率，請使用時間軸。如要進一步瞭解如何使用時間軸，請參閱 [Apple 的「讓小工具保持最新狀態」說明文件](#)。

延伸閱讀

- [iOS 應用程式擴充功能](#)
-